

Министерство высшего образования и науки Республики Казахстан
Северо-Казахстанский университет им. М. Козыбаева
Кафедра «Информационно-коммуникационные технологии»

Объектно-ориентированное программирование
МОДУЛЬ 1

**«Объектная модель учёта пациентов и медицинских приёмов в
больнице»**

Выполнили студенты группы:	ИСУ-24-1 Бахтагареев М.Р., Добош Н.А.
Проверила доцент кафедры ИКТ:	Астапенко Н.В.

Петропавловск, 2026

Содержание

1. Введение.....	2
1.1. Цель работы.....	2
1.2. Задачи.....	2
2. Анализ предметной области.....	3
2.1. Границы предметной области и основной процесс.....	3
2.2. Основные объекты и классы.....	4
2.3. Свойства и методы классов.....	4
2.4. Иерархия наследования.....	5
3. Объектная модель.....	5
3.1. Диаграмма классов.....	5
3.2. Связи между классами.....	6
3.3. Обоснование выбора типов связей.....	6
3.4. Кратности связей.....	7
3.5. Сценарий работы модели.....	7
4. Реализация модели на Python.....	9
4.1. Базовые классы и наследование.....	9
4.2. Полиморфизм.....	13
4.3. Композиция и агрегация.....	16
4.4. Ассоциации.....	18
4.5. Демонстрация работы.....	23
5. Ответы на вопросы для самопроверки.....	24
5.1. Чем отличается агрегация от композиции?.....	24
5.2. Что означает принцип наследования в объектной модели?.....	25
5.3. Какое преимущество даёт объектная модель по сравнению с процедурным подходом?.....	25
6. Заключение.....	25

Примечание: содержание построено на полях документа. После редактирования отчёта щёлкните по списку правой кнопкой мыши и выберите «Обновить поле», чтобы номера страниц пересчитались.

1. Введение

1.1. Цель работы

Цель работы — построить объектную модель предметной области «Учёт пациентов и медицинских приёмов в больнице»: выделить основные классы, определить их свойства и методы, установить между классами связи четырёх типов — обобщение, композицию, агрегацию и ассоциацию — и обосновать выбор каждого типа. Модель должна не просто перечислять объекты, а показывать, как они взаимодействуют в реальном процессе: кто создаёт медицинскую карту, кто проводит приём и куда попадает его результат. Работоспособность модели подтверждается программной реализацией на языке Python и демонстрационным сценарием, который проходит по всем связям диаграммы.

1.2. Задачи

Для достижения цели решаются следующие задачи:

1. выделить основные объекты предметной области и определить границы модели;
2. определить свойства и методы каждого класса;
3. построить диаграмму классов, используя обобщение, композицию, агрегацию и ассоциацию;
4. обосновать выбор типа и кратности каждой связи;
5. реализовать модель на Python и проверить её демонстрационным сценарием.

2. Анализ предметной области

2.1. Границы предметной области и основной процесс

Рассматривается один сквозной процесс: пациент регистрируется и получает медицинскую карту, записывается на приём к профильному врачу, медсестра готовит кабинет и сопровождает пациента, врач проводит приём и фиксирует диагноз с назначениями. Завершённый приём автоматически попадает в медицинскую карту и формирует историю обращений пациента. Так замыкается полный цикл «регистрация — приём — результат — история».

Организационная часть модели ограничена цепочкой «больница — отделения — медицинский персонал»: больница владеет отделениями, а за отделениями закреплены врачи и медсёстры. Всё, что выходит за рамки этого процесса, — лабораторные исследования, стационар, бухгалтерия, графики дежурств — в модель сознательно не включено. Задача модуля — объектная модель предметной области, а не полноценная информационная система больницы, поэтому каждая сущность должна быть оправдана сценарием, а не стремлением к «реалистичности».

2.2. Основные объекты и классы

Из описанного процесса выделяются девять классов. Человек хранит общие данные любого участника — фио, дату рождения и телефон. Пациент дополняет их полисом ОМС и собственной медицинской картой. МедицинскийПерсонал обобщает данные сотрудника — табельный номер и стаж; его наследники — Врач со специальностью, категорией и списком проведённых приёмов и Медсестра с профилем работы и сменой.

Организационную ветку образуют Больница, которая владеет отделениями, умеет искать врачей по специальности и считать статистику по сотрудникам, и Отделение, хранящее название, расположение и закреплённый персонал. Медицинскую ветку замыкают МедицинскаяКарта — документ с записями о приёмах — и Приём, центральный объект-событие: у него есть дата и время, статус, а после проведения — диагноз и назначения.

2.3. Свойства и методы классов

У каждого класса две стороны: свойства отвечают за то, что объект знает о себе, методы — за то, что он умеет делать. Состав классов сведён в таблицу 1.

Таблица 1 — Классы модели, их свойства и методы

Класс	Свойства	Методы
Человек	# фио, # датаРождения, # телефон	получитьВозраст(), получитьКонтакты()
Пациент	– полисОМС, – карта	завестиКарту(карта), записатьсяНаПриём(врач, дата), отменитьЗапись(приём, причина), получитьИсторию()
МедицинскийПерсонал	# табельныйНомер, # стажЛет	получитьИнформацию() {абстрактный}
Врач	– специальность, – категория, – приёмы	получитьИнформацию(), провестиПриём(приём, диагноз, назначения)
Медсестра	– профиль, – смена	получитьИнформацию(), подготовитьКабинет(отделе ние), сопроводитьПациента(паци ент)
Больница	– название, – адрес, – отделения	добавитьОтделение(названи е, расположение), найтиВрачей(специальност ь), получитьСтатистику()
Отделение	– название, – расположение, – персонал	закрепитьСотрудника(сотру дник), получитьРасписание()

Класс	Свойства	Методы
МедицинскаяКарта	– номер, – датаЗаведения, – записи	добавитьЗапись(приём), получитьИсторию()
Приём	– датаВремя, – статус, – диагноз, – назначения	перенести(новоеВремя), завершить(диагноз, назначения), отменить(причина), описать()

Свойства в таблице показаны с метками видимости, принятыми в UML: «#» — защищённый член, доступный наследникам, «-» — закрытый, видимый только самому классу, «+» — открытый. Все поля в реализации защищены от прямого изменения извне: имя пациента читается, но не редактируется, статус приёма меняется только его собственными методами, а списки приёмов врача и персонала отделения наружу отдаются в виде неизменяемых кортежей. Методы отражают действия, которые объект выполняет сам: пациент записывается и отменяет запись, врач проводит приём, медсестра готовит кабинет, приём переносит и отменяет сам себя.

2.4. Иерархия наследования

Иерархия наследования двухуровневая. Вершина — Человек: общие атрибуты и поведение всех участников собраны здесь, чтобы не дублировать их в наследниках. На нижнем уровне слева — Пациент, справа — МедицинскийПерсонал, у которого, в свою очередь, два наследника: Врач и Медсестра.

Человек и МедицинскийПерсонал — общие классы: в демонстрационном сценарии объекты таких типов напрямую не создаются. У МедицинскогоПерсонала метод получитьИнформацию() объявлен абстрактным — в коде он возбуждает исключение `NotImplementedError`, поэтому каждый наследник обязан дать собственную реализацию. Именно это делает вывод расписания отделения полиморфным: отделение печатает строки единым вызовом, не разбираясь, врач перед ним или медсестра.

3. Объектная модель

3.1. Диаграмма классов

Объектная модель показана на рисунке 1. Диаграмма разбита на три смысловые зоны: слева сверху — иерархия персон с четырьмя связями обобщения, справа — организационная структура больницы, внизу — медицинский процесс вокруг классов Приём и МедицинскаяКарта. Каждая связь подписана ролью и кратностью, тип связи передан стандартной нотацией UML: пустой треугольник — обобщение, закрашенный ромб — композиция, пустой ромб — агрегация, простая линия — ассоциация. Легенда внизу диаграммы поясняет обозначения.

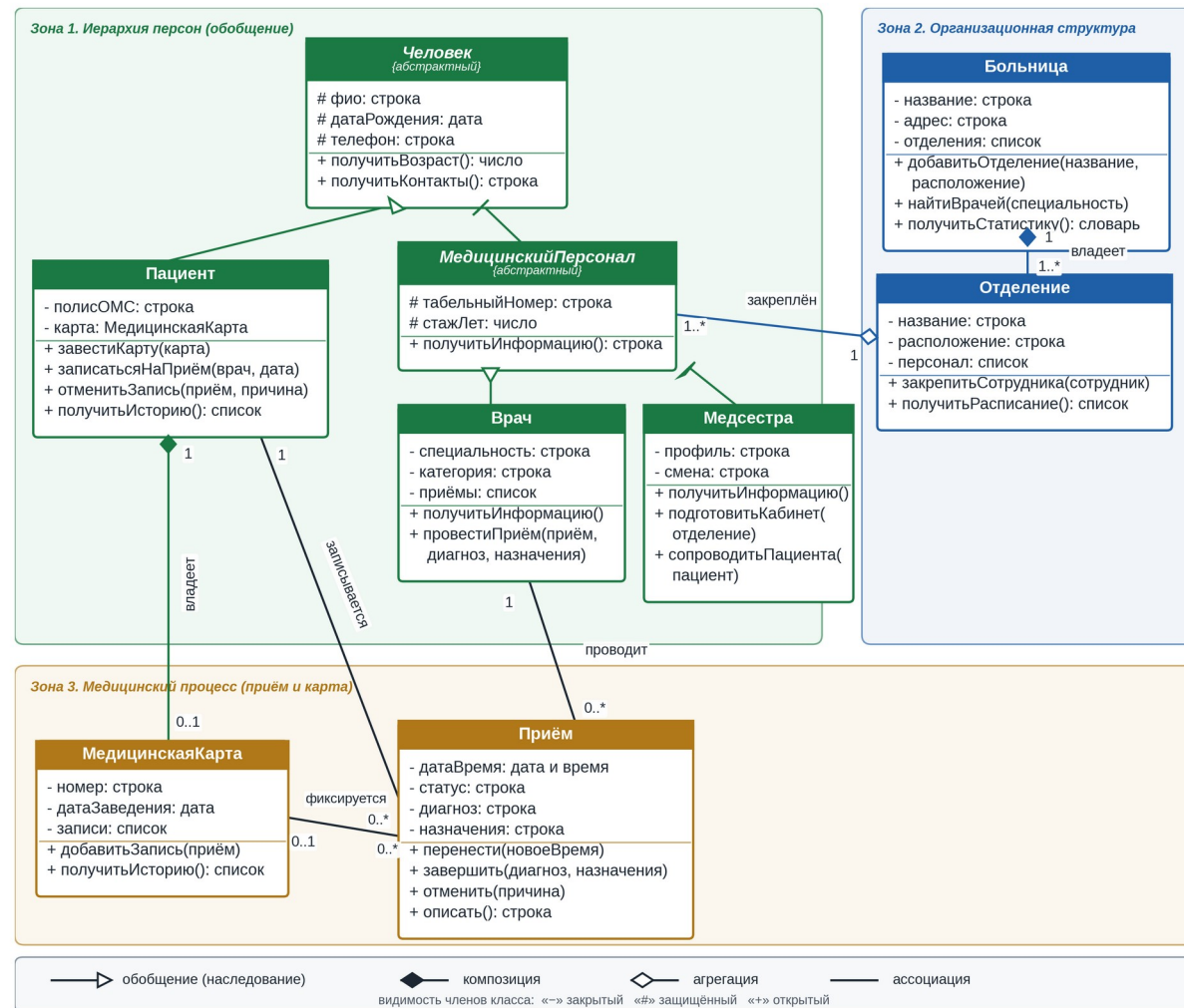


Рисунок 1 — Диаграмма классов предметной области «Учёт пациентов и приёмов в больнице»

Всего на диаграмме десять связей: четыре обобщения, две композиции, одна агрегация и три ассоциации. Зависимости (dependency) не используются сознательно: вызов метода или передача параметра не проводятся как отдельная связь, если они уже выражены ассоциацией, — иначе диаграмма превратилась бы в свалку пунктирных стрелок.

3.2. Связи между классами

Состав связей модели с типами, кратностями и обоснованиями приведён в таблице 2.

Таблица 2 — Связи между классами, их типы и кратности

Связь	Тип связи	Кратность	Обоснование
Больница — Отделение	Композиция	1 — 1..*	Отделение создаётся только методом больницы и не существует вне её; расформирование больницы уничтожает отделения вместе с ней
Пациент — МедицинскаяКарта	Композиция	1 — 0..1	Карта принадлежит только этому пациенту и заводится при регистрации; до регистрации у пациента карты нет
Отделение — МедицинскийПерсо нал	Агрегация	1 — 1..*	Сотрудники создаются вне отделения и переживают его расформирование; отделение хранит лишь ссылки на закреплённых людей
Человек — Пациент	Обобщение	—	Пациент является человеком: наследует фио, дату рождения, телефон и поведение родителя

Связь	Тип связи	Кратность	Обоснование
Человек — МедицинскийПерсонал	Обобщение	—	Сотрудник является человеком; общие данные собраны в родителе без дублирования
МедицинскийПерсонал — Врач	Обобщение	—	Врач — разновидность сотрудника со своей специализацией и поведением
МедицинскийПерсонал — Медсестра	Обобщение	—	Медсестра — разновидность сотрудника со своими действиями
Пациент — Приём	Ассоциация	1 — 0..*	Пациент записывается на приёмы; приём хранит ссылку на пациента, но не владеет им
Врач — Приём	Ассоциация	1 — 0..*	Врач проводит приёмы и ведёт список проведённых; объекты существуют независимо
МедицинскаяКарта — Приём	Ассоциация	0..1 — 0..*	Завершённый приём дописывается в карту пациента как запись об истории обращений

Каждая связь из таблицы имеет опору в коде: обобщения реализованы наследованием, композиции — созданием объекта внутри класса-владельца, агрегация — передачей готового объекта в метод закрепления, ассоциации — хранением ссылок между независимо созданными объектами. Ниже каждая связь обоснована отдельно.

3.3. Обоснование выбора типов связей

Композиция применена там, где часть не существует без целого. Отделение рождается только методом добавитьОтделение() самого объекта «Больница»: в коде конструктор Department вызывается внутри Hospital.add_department и наружу до момента создания не передаётся. Медицинская карта принадлежит ровно одному пациенту: метод завестиКарту() запрещает завести вторую, возбуждая исключение. Разница в

кратности (у больницы отделений много, у пациента карта одна) на тип связи не влияет — важно то, что жизненный цикл части полностью определяется владельцем.

Врач и медсестра создаются вне отделения и только затем закрепляются за ним методом `закрепитьСотрудника()`, который лишь добавляет готовый объект в список — ничего не конструируя. Если отделение расформировать, сотрудники не исчезнут: они существуют сами по себе и могут быть закреплены за другим отделением. Это классический признак агрегации — отношение «целое — часть» без владения жизненным циклом.

Ассоциации связывают самостоятельные объекты, которые ссылаются друг на друга, но не владеют друг другом. Пациент, врач и медицинская карта существуют до и после приёма; объект приёма лишь запоминает ссылки на участников. При этом направление связи осмысленно: приём знает своего пациента и врача, врач ведёт список проведённых приёмов, а завершённый приём сам дописывается в карту пациента вызовом `добавитьЗапись()`.

Обобщение устраняет дублирование: фео, дата рождения и телефон записаны в `Человеке` один раз и наследуются и пациентом, и персоналом; табельный номер и стаж — один раз в `МедицинскомПерсонале`. Одновременно обобщение даёт полиморфизм: расписание отделения печатает и врача, и медсестру одним и тем же вызовом `получитьИнформацию()`.

3.4. Кратности связей

Кратности в таблице 2 опираются на правила больницы и на поведение кода. Больница владеет хотя бы одним отделением — иначе объект «Больница» не имеет смысла; отделение принадлежит ровно одной больнице. За отделением закреплён хотя бы один сотрудник: отделение без персонала не работает — в демонстрации у терапевтического отделения два сотрудника, у хирургического один.

Медицинская карта помечена кратностью 0..1, а не 1: в коде поле карты у пациента первоначально пустое, и `получитьИсторию()` корректно возвращает пустой список, пока карта не заведена методом `завестиКарту()`. С пациентом связаны ноль и более приёмов — сколько раз человек обращался в больницу, столько объектов приёма и останется в системе. Каждая карта хранит 0..* записей о приёмах, при этом конкретный приём попадает максимум в одну карту — карту пациента, участвовавшего в приёме.

3.5. Сценарий работы модели

Работоспособность модели проверяется сквозным сценарием, который в точности соответствует демонстрационной программе `main.py` и проходит по всем десяти связям диаграммы.

1. создаётся больница и два отделения — терапевтическое и хирургическое: отделения рождаются внутри больницы, работает композиция «Больница — Отделение»;

2. отдельно создаются врач, хирург и медсестра, затем закрепляются за отделениями: объекты приходят извне, работает агрегация;
3. отделения печатают расписание единым вызовом `получитьРасписание()`: строки формируются полиморфным `получитьИнформацию()`, работает обобщение;
4. регистрируется пациент, ему заводится медицинская карта — композиция «Пациент — МедицинскаяКарта»;
5. пациент записывается на приём к терапевту: создаётся объект приёма со статусом «запланирован», работает ассоциация «Пациент — Приём»;
6. медсестра готовит кабинет и сопровождает пациента, врач проводит приём, ставя диагноз «ОРВИ» и назначения: приём переходит в статус «завершён», работает ассоциация «Врач — Приём»;
7. завершённый приём сам дописывается в карту пациента; второй приём переносится на другое время, третий отменяется по причине «выздоровел», после чего история обращений печатается из карты;
8. в завершение больница находит терапевтов по специальности, печатает статистику по отделениям и сотрудникам и выводит сведения о персонале единым полиморфным вызовом.

Сценарий подтверждает, что каждая связь диаграммы реально работает: композиции создают и уничтожают части вместе с владельцем, агрегация подключает внешние объекты, ассоциации соединяют независимые сущности, а обобщение обеспечивает единый полиморфный интерфейс.

4. Реализация модели на Python

4.1. Базовые классы и наследование

Модель реализована на Python в пакете `hospital_model` из шести модулей; каждый класс занимает отдельный файл. Имена классов соответствуют РЕР 8, а русским названиям с диаграммы они соответствуют напрямую: Человек — `Person`, Пациент — `Patient`, Приём — `Appointment` и так далее. Поля классов защищены одиночным подчёркиванием, что соответствует меткам «#» и «-» на диаграмме; чтение критичных данных открыто через свойства-геттеры.

```
class Person:
    """Человек – общий родитель пациента и персонала."""

    def __init__(self, full_name, birth_date, phone):
        self._full_name = full_name          # защищённое поле (UML: #)
        self._birth_date = birth_date
        self._phone = phone

    def get_age(self):
        """Возраст в полных годах."""
        today = date.today()
        age = today.year - self._birth_date.year
        if (today.month, today.day) < (self._birth_date.month,
```

```

        self._birth_date.day):
            age -= 1
        return age

class Patient(Person):
    """Пациент: полис ОМС и собственная медицинская карта."""
    # обобщение: Пациент — Человек

    def __init__(self, full_name, birth_date, phone, insurance_no):
        super().__init__(full_name, birth_date, phone)
        self._insurance_no = insurance_no
        self._card = None
        # композиция: карта пациента

    def issue_card(self, card):
        """Завести карту (у пациента она одна)."""
        if self._card is not None:
            raise ValueError("У пациента уже есть медицинская карта")
        self._card = card

```

Наследование реализовано стандартным механизмом: Patient наследует Person, MedicalStaff наследует Person, затем Doctor и Nurse наследуют MedicalStaff. Конструкторы наследников вызывают super().__init__(), передавая общие поля родителю, и добавляют только собственные. Метод get_age() вычисляет возраст в полных годах с поправкой на то, прошёл ли в текущем году день рождения человека.

4.2. Полиморфизм

Полиморфизм построен вокруг метода получитьИнформацию() (в коде — get_info()). В базовом классе МедицинскийПерсонал метод объявлен абстрактным и возбуждает NotImplementedError — это подсказка наследнику и защита от «забытой» реализации. Врач и Медсестра возвращают разные строки, а Отделение печатает расписание, не зная конкретных типов:

```

class MedicalStaff(Person):
    """Общий родитель врача и медсестры."""

    def get_info(self):
        raise NotImplementedError("Метод get_info() должен быть "
                                   "переопределён")

class Doctor(MedicalStaff):
    def get_info(self):
        # полиморфизм
        return f"Врач {self._full_name} ({self._specialty})"

class Nurse(MedicalStaff):
    def get_info(self):
        # полиморфизм
        return f"Медсестра {self._full_name} ({self._profile}, "
               f"{self._shift} смена)"

class Department:
    def schedule(self):
        """Расписание: единый формат через полиморфный get_info()."""
        return [s.get_info() for s in self._staff]

```

Тот же приём работает на уровне больницы: найтиВрачей() перебирает персонал всех отделений, отбирая врачей проверкой isinstance, а получитьСтатистику() считает врачей и медсестёр по одному общему интерфейсу и возвращает словарь со сводкой.

4.3. Композиция и агрегация

Композиция в коде видна по месту создания объекта. Отделение конструируется внутри метода больницы и сразу попадает в её список:

```
class Hospital:
    def add_department(self, name, location):
        """Отделение создаётся внутри больницы (композиция)."""
        dept = Department(name, location)
        self._departments.append(dept)
        return dept

class Department:
    def attach_staff(self, person):
        """Закрепить сотрудника за отделением (не создаёт его!)."""
        self._staff.append(person)           # агрегация
```

Медицинская карта заводится пациенту методом завестиКарту(), показанным в разделе 4.1: попытка завести вторую перехватывается исключением, поэтому у пациента не может появиться две карты. Агрегация устроена иначе: закрепитьСотрудника() принимает готовый объект и только добавляет его в список отделения, ничего не создавая — в демонстрации врач и медсестра создаются до того, как отделения вообще узнают об их существовании.

4.4. Ассоциации

Ассоциации соединяют независимо созданные объекты ссылками. Запись на приём инициирует пациент, а создаёт объект приёма его собственный метод — приём с первого момента существования знает обоих участников:

```
class Patient:
    def make_appointment(self, doctor, when):
        """Записаться на приём: возвращает объект Приёма."""
        return Appointment(when, self, doctor)

class Doctor:
    def conduct_appointment(self, appointment, diagnosis,
                             prescription=None):
        """Провести приём: диагноз и назначения."""
        appointment.complete(diagnosis, prescription)
        self._appointments.append(appointment)

class Appointment:
    def complete(self, diagnosis, prescription=None):
        """Завершить приём; запись попадает в карту пациента."""
        if self._status != "запланирован":
            raise ValueError("Завершить можно только запланированный")
```

```

        " приём")
self._status = "завершён"
self._diagnosis = diagnosis
self._prescription = prescription
if self._patient.card is not None:
    self._patient.card.add_record(self) # ассоциация с картой

def reschedule(self, new_when):
    """Перенести приём (отменённый переносить нельзя)."""
    if self._status == "отменён":
        raise ValueError("Нельзя перенести отменённый приём")
    self._when = new_when

```

Проведение приёма — работа врача: приём завершается, диагноз и назначения сохраняются, приём попадает в список врача и автоматически дописывается в карту пациента. Состояние приёма защищено правилами: перенести отменённый приём нельзя, отменить завершённый — тоже; за это отвечают проверки в методах самого приёма, поэтому недопустимый переход статуса невозможно выполнить в обход модели.

4.5. Демонстрация работы

Демонстрационная программа main.py проходит сценарий из раздела 3.5 по всем восьми шагам. Ниже приведён фрагмент её вывода, относящийся к ключевым связям модели:

```

Записан на приём: 20.09.2026 10:30 — Врач Иванов А.С. (терапевт) —
запланирован
Смирнова Е.П. готовит кабинет (Терапевтическое)
Приём проведён: 20.09.2026 10:30 — Врач Иванов А.С. (терапевт) —
завершён —
    диагноз: ОРВИ — назначения: Постельный режим, обильное питьё
Перенесён: 26.09.2026 09:30 — Врач Петров В.К. (хирург) — запланирован
Отменён: 01.10.2026 11:00 — Врач Иванов А.С. (терапевт) — отменён
(выздоровел)
История обращений пациента (из медицинской карты):
    * 20.09.2026 10:30 — Врач Иванов А.С. (терапевт) — завершён — диагноз:
ОРВИ
Статистика:
    Отделений: 2      Сотрудников: 3      Врачей: 2      Медсестёр: 1

```

Вывод подтверждает согласованность модели и кода: расписание печатается полиморфно, статус приёма меняется только его методами, история читается из карты, а статистика собирается по всей больнице.

5. Ответы на вопросы для самопроверки

5.1. Чем отличается агрегация от композиции?

Оба отношения описывают связь «целое — часть», но различаются управлением жизненным циклом части. При композиции часть не существует отдельно от целого: она создаётся внутри владельца и исчезает вместе с ним. При агрегации часть создаётся независимо, приходит к владельцу готовой и переживает его исчезновение.

В построенной модели обе ситуации показаны явно. Отделение создаётся только методом больницы и не существует вне её — это композиция. Врач и медсестра создаются сами по себе и лишь закрепляются за отделением; расформирование отделения их не уничтожает — это агрегация. В коде разница видна по месту создания объекта: конструктор внутри метода владельца даёт композицию, приём готового объекта извне — агрегацию.

5.2. Что означает принцип наследования в объектной модели?

Наследование — это отношение «является», при котором дочерний класс получает атрибуты и методы родителя, добавляя собственные или переопределяя унаследованные. В объектной модели оно выражается связью обобщения — стрелкой с пустым треугольником от наследника к родителю.

В модели наследование двухуровневое: Пациент и МедицинскийПерсонал наследуют у Человека общие данные и поведение, Врач и Медсестра — у МедицинскогоПерсонала табельный номер и стаж. Практический смысл — устранение дублирования и полиморфизм: общий код пишется один раз в родителе, а единый метод получитьИнформацию() каждый наследник наполняет своим содержанием.

5.3. Какое преимущество даёт объектная модель по сравнению с процедурным подходом?

При процедурном подходе данные и функции их обработки разделены: любая функция получает «сырые» записи и сама следит за их корректностью. Объектная модель соединяет данные и поведение в классе: состояние изменяют только методы самого объекта, поэтому недопустимое состояние — например, отмена уже завершённого приёма — нельзя создать в обход правил.

Для построенной модели это дало три конкретных преимущества. Инкапсуляция: статус приёма, номер карты и списки приёмов врача защищены от внешнего изменения. Полиморфизм: расписание отделения печатается одним вызовом для любых типов сотрудников. Расширяемость: новый тип сотрудника, например лаборант, добавляется наследником МедицинскогоПерсонала, и расписание отделения начнёт выводить его без изменения собственного кода.

6. Заключение

В работе построена объектная модель предметной области «Учёт пациентов и медицинских приёмов в больнице». Выделены девять классов, для каждого определены свойства и методы, модель сведена к диаграмме классов с десятью связями: четырьмя обобщениями, двумя композициями, одной агрегацией и тремя ассоциациями. Зависимости на диаграмме не используются: тип каждой связи обоснован жизненным циклом объектов и подтверждён местом создания объекта в коде.

Модель согласована с реализацией: каждый класс диаграммы имеет реализацию в пакете `hospital_model`, каждая связь — проверяемое доказательство в коде, а демонстрационный сценарий проходит по всем связям без ошибок. Построенная модель пригодна к развитию: новые типы сотрудников и новые виды документов добавляются наследованием и ассоциацией, не ломая существующую структуру.